PRO2000 - PROGRAMMING

Week 44- Parallel programming

Gebremariam Assres
Gebremariam.assres@kristiania.no

[Adapted from QwertyPi (Daniel Hsieh)]

Today's topics

- Why parallel programming?
- Processes and threads
- Parallel processing
- Race condition
- Mutual exclusion (mutex)
- Synchronization methods
- Design principles

Why parallel programming?

- How to speed up processing?

- Increase hardware clock rate
- Issue: heat dissipation

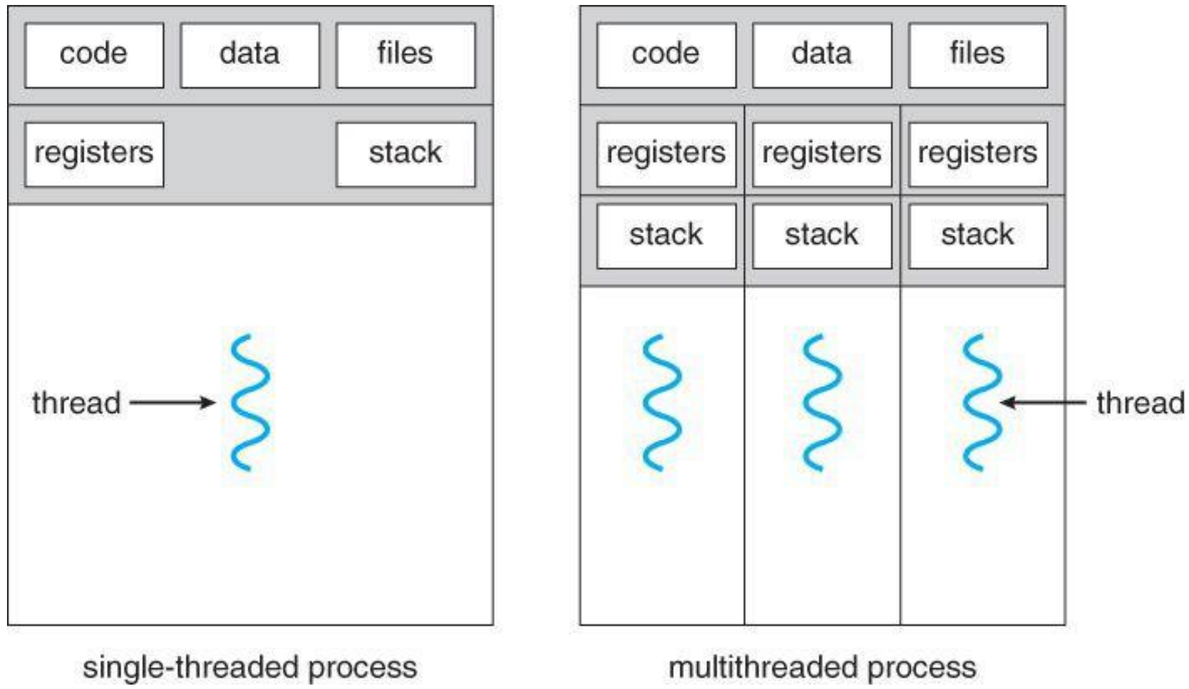
- ⇒**Run in parallel**



Source: Wikipedia

Processes and threads

- A process is an independent program running in memory, while a thread is a smaller unit of execution inside that process.
- Threads are shared memory:
  - Shared (common to all threads in the process): global variables (bss), heap memory, and program code.
  - Isolated (private to each thread): registers, program counter, and local (stack) variables.
  - We focus on multi-threading today



single-threaded process

multithreaded process

Parallel processing

- Example: Sum of Array in Parallel
  - This problem is parallel: so, we can split it into several unrelated tasks easily
  - Target: Compute the sum of $n=10^8$ integers

```
long long global_sum = 0;
for (int i = 0; i < n; i++) {
    global_sum += a[i];
}
```

```
Sum = 21473786176982382
Time Elapsed = 224864µs
```

3.95x

```
long long global_sum = 0;
mutex sum_mutex;
void compute_sum(vector<int>& a, int start, int end) {
    long long sum = 0;
    for (int i = start; i < end; i++) sum += a[i];
    lock_guard<mutex> lock(sum_mutex);
    global_sum += sum;
}
int num_threads = 4;
vector<thread> threads;
int chunk_size = (n + num_threads - 1) / num_threads;
for (int t = 0; t < num_threads; t++) {
    int start = t * chunk_size;
    int end = min(start + chunk_size, n);
    threads.emplace_back(compute_sum, ref(a), start, end);
}
for (auto& th : threads) {
    th.join();
}
```

```
Sum = 21473786176982382
Time Elapsed = 56925µs
```

Parallel processing ...

- Example: Sum of Array in Parallel ...

2. "**Locks**" the global sum

1. Constructs and **spawns** the thread

3. Waits for the thread to **end and join**

```
long long global_sum = 0;
mutex sum_mutex;
void compute_sum(vector<int>& a, int start, int end) {
    long long sum = 0;
    for (int i = start; i < end; i++) sum += a[i];
    lock_guard<mutex> lock(sum_mutex);
    global_sum += sum;
}
int num_threads = 4;
vector<thread> threads;
int chunk_size = (n + num_threads - 1) / num_threads;
for (int t = 0; t < num_threads; t++) {
    int start = t * chunk_size;
    int end = min(start + chunk_size, n);
    threads.emplace_back(compute_sum, ref(a), start, end);
}
for (auto& th : threads) {
    th.join();
}
```

Race condition

- Example: Sum of Array in Parallel - Data Race

What if no lock, and we directly add to global_sum?

```
Sum = 67425740374814124
Time Elapsed = 383729µs
```

The sum is smaller than expected…
Also, it is (comparatively) super slow

Exemplar of **data races**

```cpp
long long global_sum = 0;
//mutex sum_mutex;
void compute_sum(vector<int>& a, int start, int end) {
    //long long sum = 0;
    for (int i = start; i < end; i++) sum += a[i];
    //lock_guard<mutex>lock(sum_mutex);
    //global_sum += sum;
}
int num_threads = 4;
vector<thread> threads;
int chunk_size = (n + num_threads - 1) / num_threads;
for (int t = 0; t < num_threads; t++) {
    int start = t * chunk_size;
    int end = min(start + chunk_size, n);
    threads.emplace_back(compute_sum, ref(a), start, end);
}
for (auto& th : threads) {
    th.join();
}
```

Race condition ...

```
BSS             global_cnt = 0;
```

```
Thread 1
int N = 100000;
for (int i = 0; i < N; i++) {
   global_cnt++;
}
```

```
Thread 2
int N = 100000;
for (int i = 0; i < N; i++) {
   global_cnt++;
}
```

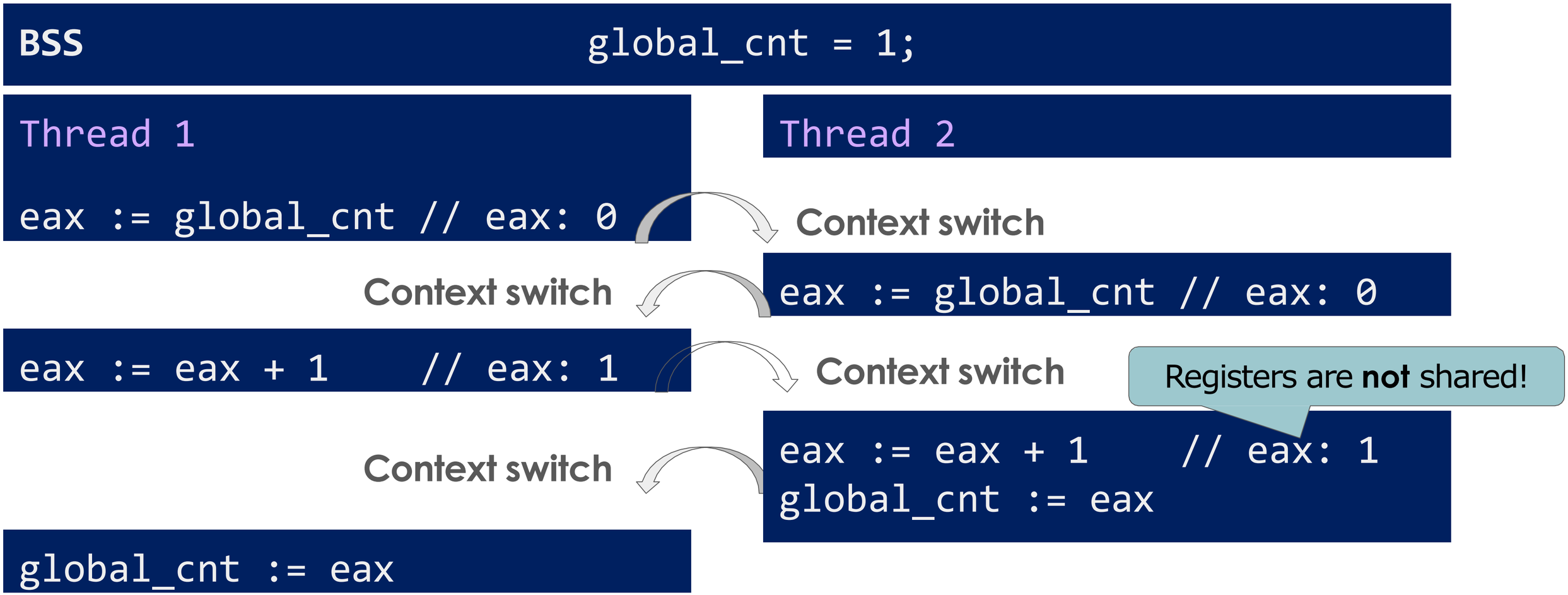
Expect: 200000
Actual: 117963

Why?

Incrementing **global_cnt++** translates to three non atomic operation is (using registers as in the assembly code below):

```
mov eax, [global_cnt]       ; eax := global_cnt
inc eax                     ; eax := eax + 1
mov [global_cnt], eax       ; global_cnt := eax
```

Race condition ...



Race condition …

```
BSS             global_cnt = 1;
```

```
Thread 1
eax := global_cnt // eax: 0

eax := eax + 1 // eax: 1



global_cnt := eax
```

```
Thread 2

eax := global_cnt // eax: 0

eax := eax + 1 // eax: 1
global_cnt := eax
```

An **instruction interleaving** where global_cnt has a final value of 1!

Race condition ...

- Data races occur when two threads:
  - Access the **same *non-atomic* memory location** simultaneously.
  - At least one of the accesses is a **write**.
  - There is **no synchronization** between the two threads.

- In C++, data races will lead to **undefined behaviour**.

Mutual exclusion

- Mutual Exclusion: Failed Attempt
  - We have to avoid both threads simultaneously accessing global_cnt.
  - What about adding another variable?

```
if (!in_use) {
  in_use = true;
  global_cnt++;
  in_use = false;
}
```

Mutual exclusion ...

- Mutual Exclusion: Failed Attempt

```
BSS                 in_use = false;          global_cnt = 0;
```

```
Thread 1

if (!in_use)          // TRUE
```



Context switch

```
Thread 2

if (!in_use)          // TRUE
```

```
in_use = true;
global_cnt++;
in_use = false;
```

```
in_use = true;
global_cnt++;
in_use = false;
```

Mutual exclusion ...

- Mutual Exclusion in C++
  - Threads should not access the same memory at the same time
  - Hence called **Mut**ual **Ex**clusion (mutext)
  - C++ mutex: lock() and unlock()
  - Once one thread locks the mutex, the other waits for it to unlock, avoiding data race

```
mutex cnt_mutex;
int N = 100000;
for (int i = 0; i < N; i++) {
    cnt_mutex.lock();
    global_cnt++;
    cnt_mutex.unlock();
}
// global_cnt = 200000
```

Mutual exclusion ...

- Mutual Exclusion in C++

```
BSS          cnt_mutex: Locked;          global_cnt = 0;
```

```
Thread 1

cnt_mutex.lock();
```



Context switch

```
Thread 2
```

Mutex is already locked
⇒ The thread **blocks**.

```
cnt_mutex.lock();
```

Mutual exclusion ...

- Mutual Exclusion in C++

```
BSS          cnt_mutex: Unlocked;        global_cnt = 1;
```

```
Thread 1

cnt_mutex.lock();
```

```
Thread 2
```

Context switch



```
cnt_mutex.lock();
```

```
global_cnt++;
cnt_mutex.unlock();
```

Mutual exclusion ...

- Mutual Exclusion in C++

```
BSS          cnt_mutex: Locked;          global_cnt = 1;
```

```
Thread 1

cnt_mutex.lock();
```

```
Thread 2
```

The thread **wakes up** and locks the mutex!

```
cnt_mutex.lock();
```

```
global_cnt++;
cnt_mutex.unlock();
```

Mutual exclusion ...

- Mutual Exclusion in C++

```
BSS          cnt_mutex: Unlocked;      global_cnt = 2;
```

```
Thread 1

cnt_mutex.lock();
```

```
Thread 2
```

```
cnt_mutex.lock();
```

```
global_cnt++;
cnt_mutex.unlock();
```

```
global_cnt++;
cnt_mutex.unlock();
```

Synchronization methods

- RAII (Resource Acquisition Is Initialisation)

Questions:
- Why constructor spawns thread?
- Why don't we need to "unlock"?

- Ties resources to object lifetime
- Constructor: acquire resources
- Destructor: release resources
- C++ does RAII by default for automatic storage (stack)
- No need manually manage memory that much anymore!

```
long long global_sum = 0;
mutex sum_mutex;
void compute_sum(vector<int>& a, int start, int end) {
    long long sum = 0;
    for (int i = start; i < end; i++) sum += a[i];
    lock_guard<mutex> lock(sum_mutex);
    global_sum += sum;
}
int num_threads = 4;
vector<thread> threads;
int chunk_size = (n + num_threads - 1) / num_threads;
for (int t = 0; t < num_threads; t++) {
    int start = t * chunk_size;
    int end = min(start + chunk_size, n);
    threads.emplace_back(compute_sum, ref(a), start, end);
}
for (auto& th : threads) {
    th.join();
}
```

Synchronization methods ...

- Mutual Exclusion in C++ with RAII
  - Manually handling lock is not good
  - You may forget to unlock
  - Unlock is needed even when exception raised
  - With RAII: lock_guard<mutex>
  - Unlock itself once goes out of scope

```
mutex cnt_mutex;
int N = 100000;
for (int i = 0; i < N; i++) {
    // cnt_mutex.lock();
    lock_guard<mutex>
        lock(cnt_mutex);
    global_cnt++;
    // cnt_mutex.unlock();
}
```

Synchronization methods ...

- Semaphores
  - Extension of mutex (C++20)
  - Allows at most N concurrent connections
  - Example: Queue for parking space, wait to enter a store

Decrease count if success

Increase count

```
counting_semaphore<size_t> sem(5);
if (sem.try_acquire()) {
    // critical section
    sem.release();
}
```

Synchronization methods ...

- Barriers
  - Sometimes you want first complete all of step 1, then step 2
  - Barrier: wait for threads to finish, then starts again

```
// spawn threads for step 1
for (int t = 0; t < num_threads; t++) {
    threads.emplace_back(step_1, t);
}
// barrier: wait for all threads to complete
for (auto& th : threads) {
    th.join();
}
// then continue step 2
for (int t = 0; t < num_threads; t++) {
    threads.emplace_back(step_2, t);
}
for (auto& th : threads) {
    th.join();
}
```

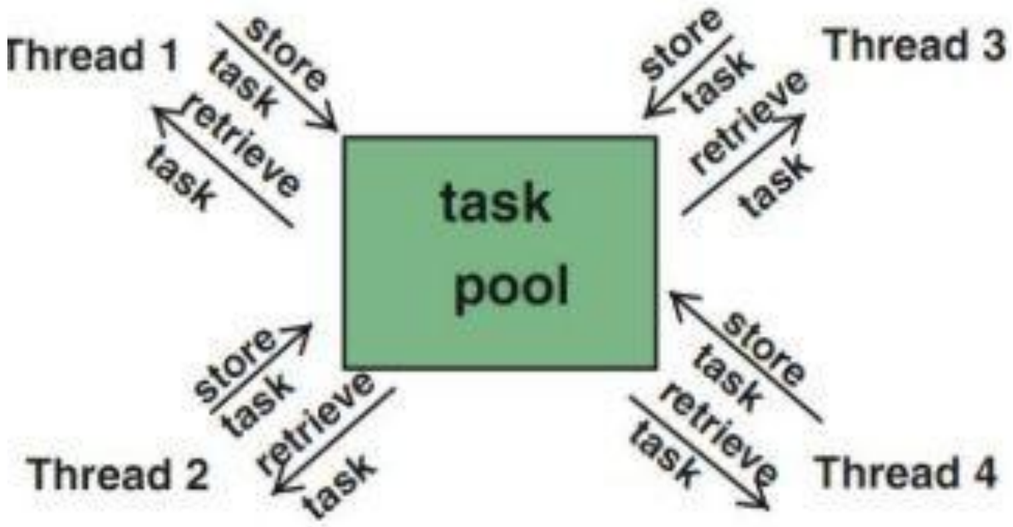
Synchronization methods …

- Atomic operations
  - You can declare stuff as “atomic” (C++11)
  - Operations are done as an entirety
  - Won’t mess up even if multi-threaded
  - Example: maintain counter in multi-threaded program

```
atomic<int> acnt;
void increment_counter() {
    for (int n = 10000; n; --n) {
        ++acnt;
    }
}
```

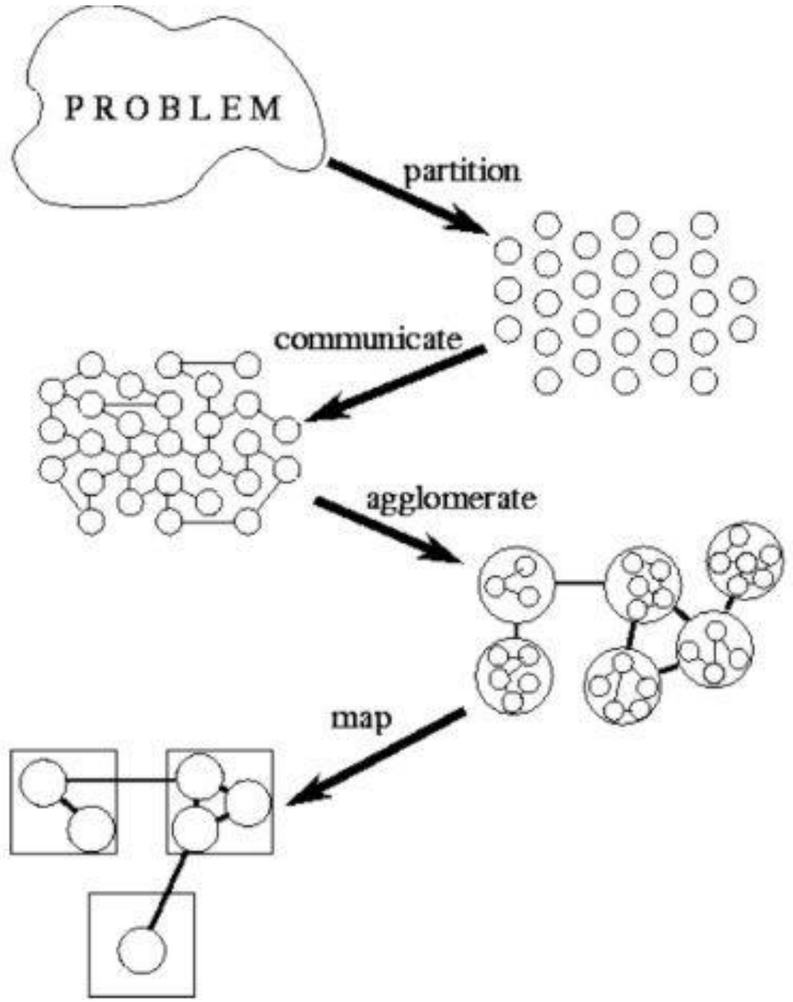
Design principles

- How to design parallel algorithms?
  - Instantiate a "pool of tasks"
  - Once a thread is done with it's job, it takes another job.
  - So, task distribution goes on like this …



Design principles ...

- How to design parallel algorithms ...
  1. **Partition** the problem into small tasks
  2. Small tasks **communicate** with each other
  3. **Agglomerate** tasks to groups to reduce communication cost
  4. **Map** tasks to processors (cores)



Up next: Exercises

- Now: 2 hours (12:15 to 14:00) of exercises (found on Canvas).

- Short exercise summary:
  - *Write, compile, and debug a C++ programs with emphasis on parallel programing*
- Remember:
  - You should do approximately 4 more hours of the course (in a study group or alone) *before* our next session together. :-)